

Compiler Design

Chapter 5

Intermediate code generation

Chapter Six : Intermediate Code Generation

- Intermediate Languages
- Declarations

Chapter Six : Intermediate Code Generation

- Intermediate code can translate the source program into the machine program.
- Intermediate code is generated because the compiler can't generate machine code directly in one pass.
- Therefore, first, it converts the source program into intermediate code, which performs efficient generation of machine code further.
- The intermediate code can be represented in the form of *postfix notation, syntax tree, directed acyclic graph, three address codes, Quadruples, and triples.*

Chapter Six : Intermediate Code Generation

Postfix Notation

- The postfix notation is practical for an intermediate representation as the operands are found just before the operator
- In fact, the postfix notation is a linearized representation of a syntax tree
- Examples

*Input: $A + B * C + D$*

Output: $ABC+D+$*

*Input: $((A + B) - C * (D / E)) + F$*

Output: $AB+CDE/-F+$*

Chapter Six : Intermediate Code Generation

Syntax Tree

- A syntax tree is nothing more than a condensed form of a parse tree.
- The operator and keyword nodes of the parse tree are moved to their parents and a chain of single productions is replaced by the single link in the syntax tree the internal nodes are operators and child nodes are operands.
- To form a syntax tree put parentheses in the expression, this way it's easy to recognize which operand should come first.

Chapter Six : Intermediate Code Generation

Syntax Tree

○ Example

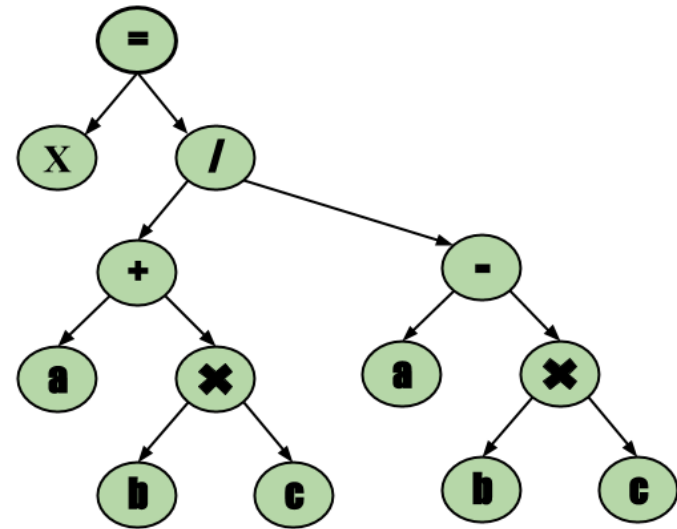
$X = A + B$

$X = A * B - C$

$x = (a + b * c) / (a - b * c)$

$X = (a + (b * c)) / (a - (b * c))$

Operator Root



Chapter Six : Intermediate Code Generation

Three address Code

- The three address code is a sequence of statements of the form: $X := Y \text{ op } Z$
- Where: X, Y, and Z are names, constants or compiler-generated temporaries **op** is an operator such as integer or floating point arithmetic operator or logical operator on Boolean data.

Chapter Six : Intermediate Code Generation

Three address Code

○ Important Notes:

- No built-up arithmetic operator is permitted
- Only one operator at the right side of the assignment is possible, i.e., $x + y + z$ is not possible.
- Similarly to postfix notation, the three-address code is a linearized representation of a syntax tree.
- It has been given the name three-address code because such an instruction usually contains three-addresses (the two operands and the result).

Chapter Six : Intermediate Code Generation

Types of Three-Address Statements

- As with an assembler statement, the three-address code statement can have: symbolic labels, as well as statements for control flow.

Chapter Six : Intermediate Code Generation

Types of Three-Address Statements

○ Assignment (binary operation) $a := b \text{ op } c$

- a, b and c are the operands.
- Operands may be constants, names, or compiler generated temporaries.
- op represents the operator.

○ Assignment (unary operation) $x := \text{op } y$ $x := y \text{ op } z$

- x, y and z are the operands.
- op represents the operator.
- It assigns the result obtained after solving the right side expression of the assignment operator to the left side operand.

Chapter Six : Intermediate Code Generation

Types of Three-Address Statements

○ Copy

$x := y$

- x and y are the operands.
- = is an assignment operator.
- It copies and assigns the value of operand y to operand x.

○ Unconditional Jump

Goto X

- Here, X is the tag or label of the target statement.
- On executing the statement,
- The control is sent directly to the location specified by label X.
- All the statements in between are skipped.

Chapter Six : Intermediate Code Generation

Types of Three-Address Statements

○ Conditional Jump if x relop y goto L

- x & y are the operands.
- L is the tag or label of the target statement.
- relop is a relational operator.
- If the condition “x relop y” gets satisfied, then-
 - The control is sent directly to the location specified by label L.
 - All the statements in between are skipped.
- If the condition “x relop y” fails, then-
 - The control is not sent to the location specified by label L.
 - The next statement appearing in the usual sequence is executed.

Chapter Six : Intermediate Code Generation

Types of Three-Address Statements

○Function call

○Parameter specification

- param x1

○Calling the function

- call P, N { P : function & N : no. of parameters }

○Indexed Arguments

- $x := y[I]$ { x = value at address $y+I$ }
- $y[I] := x$ { value at address $y+I = x$ }

Chapter Six : Intermediate Code Generation

Types of Three-Address Statements

○ Address and Pointer Assignments

○ $x := \&y$ { x = address of y }

○ $x := *y$ { x = the value at address y }

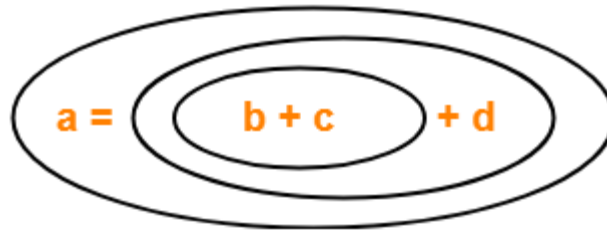
○ $*x := y$ { value at address x = y }

Chapter Six : Intermediate Code Generation

Types of Three-Address Statements

○ Examples Represent address three statement

$a = b + c + d$

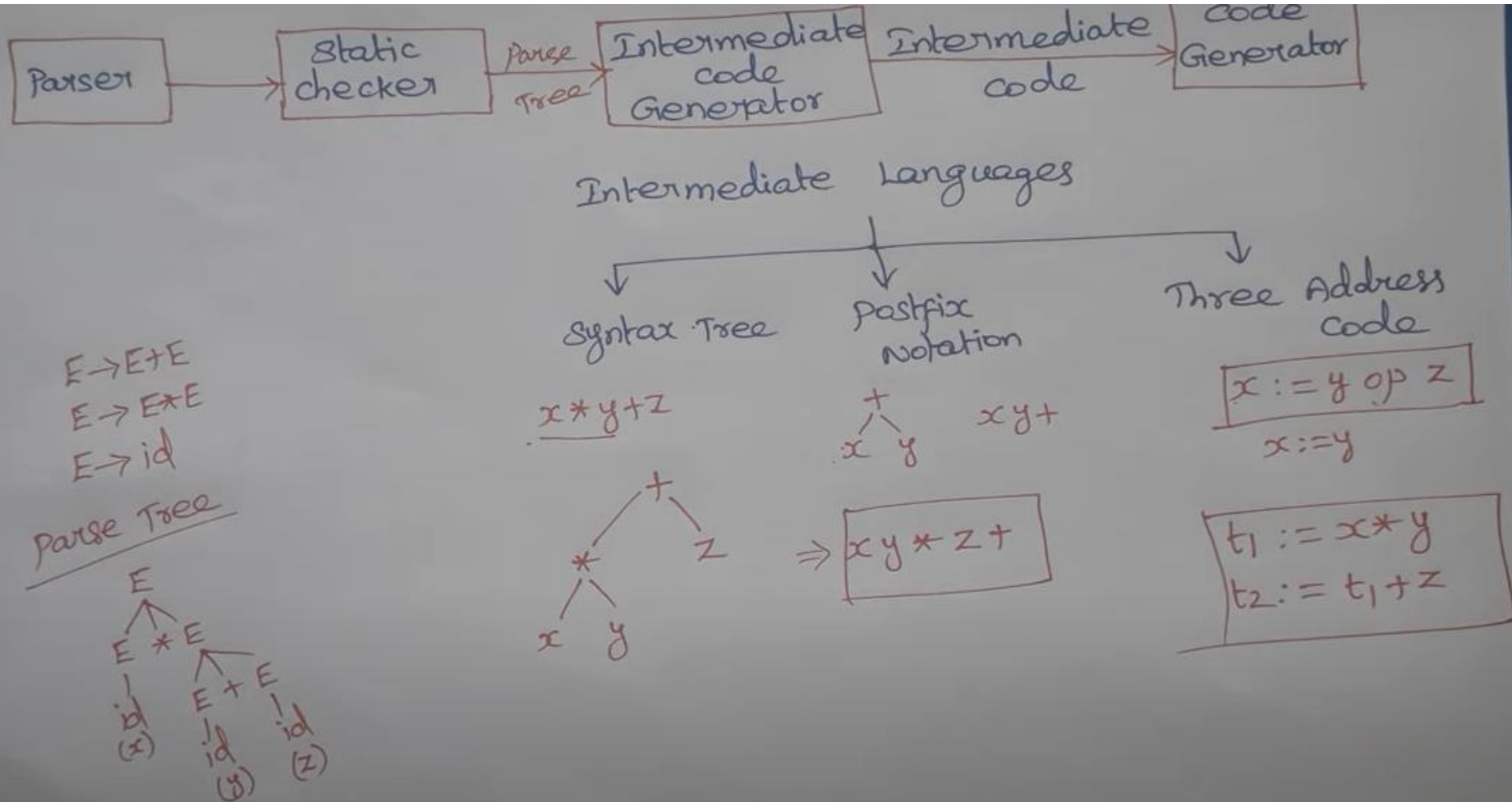


(1) $T1 = b + c$

(2) $T2 = T1 + d$

(3) $a = T2$

Chapter six : Intermediate Code Generation



Chapter Six : Intermediate Code Generation

Types of Three-Address Statements

○ Examples Represent address three statement

$$-(a * b) + (c + d) - (a + b + c + d)$$

• Three Address Code for the given expression is-

- | | |
|--------------------------------|-----------------|
| • (1) $T1 = a * b$ | $T1 := a * b$ |
| • (2) $T2 = \text{uminus } T1$ | $T2 := -T1$ |
| • (3) $T3 = c + d$ | $T3 := c + d$ |
| • (4) $T4 = T2 + T3$ | $T4 := T2 + T3$ |
| • (5) $T5 = a + b$ | $T5 := a + b$ |
| • (6) $T6 = T3 + T5$ | $T6 := T3 + T5$ |
| • (7) $T7 = T4 - T6$ | $T7 := T4 - T5$ |

Chapter Six : Intermediate Code Generation

Syntax Directed Translation into Three Address Code

- The 3 address code is generated either as an attribute of the parse tree or by writing the statements in a file as soon as they are generated.
- When the 3 address code is generated, it is often necessary to use temporary variables and temporary names.
- For this purpose, the following functions are given:

Chapter Six : Intermediate Code Generation

Syntax Directed Translation into Three Address Code

- NewTemp() : each time it is called, gives distinct names that can be used for temporary variables.
- NewLabel() : each time it is called gives distinct names that can be used for label names.
- In addition, we use the function **gen** to create three address code statements from a number of strings.

Chapter Six : Intermediate Code Generation

Syntax Directed Translation into Three Address Code

Production	Semantic rule
$S \rightarrow id = E$	<code>{id.place = lookup(id.name); if id.place $\neq$ null then S.code = E.code gen(id.place ":=" E.place) else S.code = type_error}</code>
$E \rightarrow - E_1$	<code>{E.place = newtemp(); E.code = E₁.code gen(E.place ":=" "-" E₁.place)}</code>
$E \rightarrow E_1 + E_2$	<code>{E.place = newtemp(); E.code = E₁.code E₂.code gen(E.place ":=" E₁.place "+" E₂.place)}</code>
$E \rightarrow E_1 - E_2$	<code>{E.place = newtemp(); E.code = E₁.code E₂.code gen(E.place ":=" E₁.place "-" E₂.place)}</code>
$E \rightarrow E_1 * E_2$	<code>{E.place = newtemp(); E.code = E₁.code E₂.code gen(E.place ":=" E₁.place "*" E₂.place)}</code>
$E \rightarrow E_1 / E_2$	<code>{E.place = newtemp(); E.code = E₁.code E₂.code gen(E.place ":=" E₁.place "/" E₂.place)}</code>
$E \rightarrow id$	<code>{E.place = lookup(id.name), E.code = " "}</code>

Chapter Six : Intermediate Code Generation

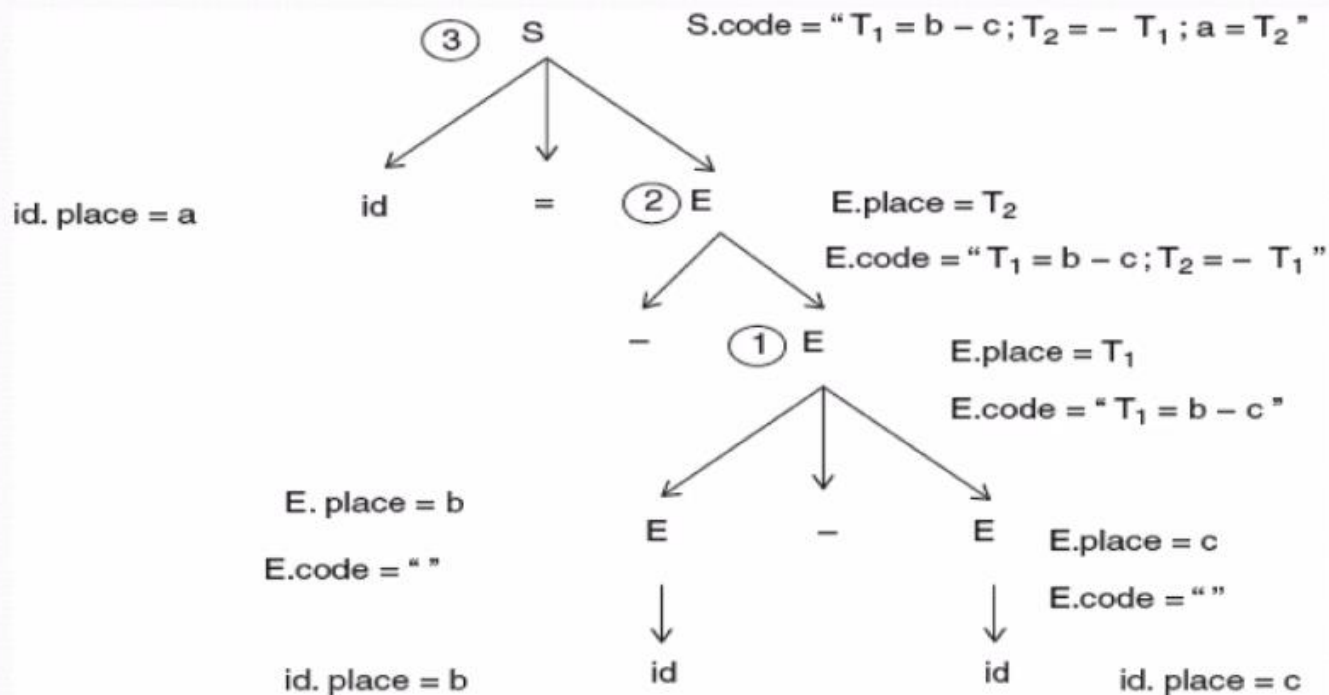
Syntax Directed Translation into Three Address Code

- The attribute *place* will hold the value of the grammar symbol
- The attribute *code* will hold the sequence of three-address statements evaluating the grammar symbol
- The function *newtemp* returns a sequence of distinct names $t_1, t_2, \dots$ in response to successive calls

Chapter Six : Intermediate Code Generation

Syntax Directed Translation into Three Address Code

Example :- Syntax tree for $a = -(b - c)$



Chapter Six : Intermediate Code Generation

Syntax Directed Translation into Three Address Code

$$-(a * b) + (c + d) - (a + b + c + d)$$

Exercise

$$X = (a+b) - (c+d) * (a+b+c+d)$$

1. Three address representation
2. Postfix representation
3. Syntax tree
4. Quadruple
5. Triple
6. Indirect Triple
7. Direct acyclic graph
8. Optimize the above three address code
9. Generate target code for the above expression (assembly code)
10. Optimize the assembly code you have got from Q9.